



Manipulación de datos y automatización con R

Manipulación de datos con dplyr, tidyr y lubridate

Mg. J. Eduardo Gamboa U.

2026-07-13

Introducción

- ▶ Datos = diamante en bruto
- ▶ **Garbage in, garbage out**
- ▶ Tareas de preparación o preprocesamiento:
 - ▶ Limpieza
 - ▶ Transformación
 - ▶ Integración
- ▶ Haremos uso de los paquetes **dplyr** y **magrittr**. Este último permite añadir el operador pipe `%>%` (Control + Shift + M en RStudio)
- ▶ No obstante, también es posible utilizar el pipe nativo `|>`

DATA ANALYSIS

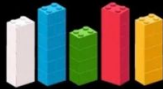
1 DATA



2 SORTED



3 ARRANGED



4 PRESENTED VISUALLY



5 EXPLAINED WITH A STORY



Funcionamiento del pipe

El operador pipe permite que dispongamos los elementos de una función de manera opuesta, permitiendo la concatenación. Por ejemplo:

```
library(dplyr)
x = c(2,4,5,3)
sum(x)
```

```
[1] 14
```

```
x %>% sum
```

```
[1] 14
```

```
x |> sum()
```

```
[1] 14
```

En general $f(x)$ es expresado como $x \%>\% f$.

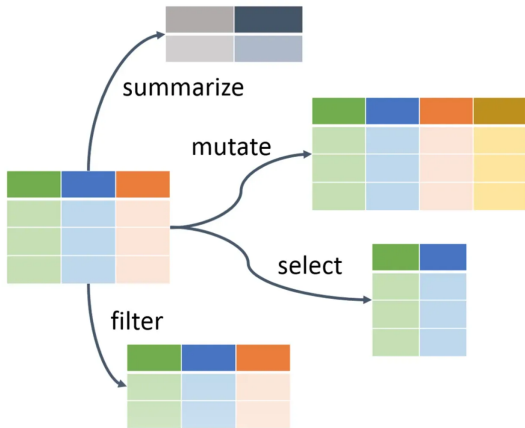
Tiene más utilidad cuando se usa una serie de funciones anidadas, por ejemplo:

$h(g(f(x)))$ es equivalente a $x \%>\% f \%>\% g \%>\% h$ o $x \%>\% f() \%>\% g() \%>\% h()$

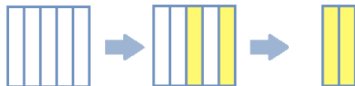
A partir de la versión 4.1.0 (mayo 2021), existe un nuevo pipe en R: $|>$

Es decir $h(g(f(x)))$ es equivalente a $x |> f() |> g() |> h()$

Paquete dplyr



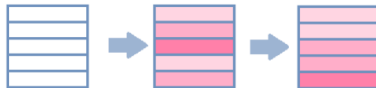
select



filter



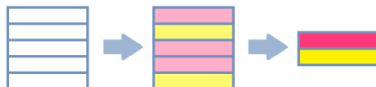
arrange

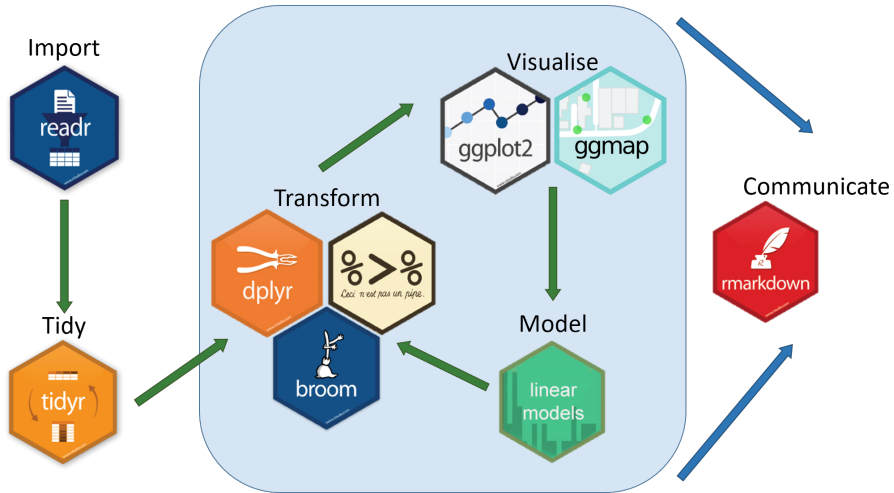


mutate



summarise





```
A <- c(12, 20, NA, 14, 13, NA, 17, 11, 9, NA)
B <- c(NA, NA, 13, 7, NA, 16, 19, NA, 19, 10)
C <- c(11, NA, 5, 4, 8, 16, NA, 8, 19, 10)
Categoria <- c("Alfa", "Beta", "Alfa", "Gamma", "Beta", "Gamma",
               "Alfa", "Gamma", "Beta", "Alfa")
Grupo <- c("X", "Y", "X", "Z", "Y", "Z", "X", "Z", "Y", "X")
datos <- data.frame(A, B, C, Categoria, Grupo)
datos
```

	A	B	C	Categoria	Grupo
1	12	NA	11	Alfa	X
2	20	NA	NA	Beta	Y
3	NA	13	5	Alfa	X
4	14	7	4	Gamma	Z
5	13	NA	8	Beta	Y
6	NA	16	16	Gamma	Z
7	17	19	NA	Alfa	X
8	11	NA	8	Gamma	Z
9	9	19	19	Beta	Y
10	NA	10	10	Alfa	X

Funciones para preparación de datos

- ▶ `select` permite seleccionar o retirar una o más columnas de un data frame.
- ▶ `pull` permite seleccionar una columna de un data frame.
- ▶ `mutate` permite crear una nueva columna de un data frame o modificar una existente
- ▶ `rename` permite renombrar una o más columnas de un data frame.
- ▶ `filter` permite filtrar los valores de una variable según una o más reglas.
- ▶ `arrange` ordena las filas de un data frame según los valores de una o más columnas

Función: `select`

Paquete: `dplyr`

¿Cómo se usa?

```
df |> select(posiciones o nombres de las variables a ser  
seleccionadas)
```

```
df |> select(-posiciones o nombres de las variables a ser eliminadas)
```

```
df |> select(condición)
```

Ejemplos

1. Seleccionar la columna DEPARTAMENTO.
2. Seleccionar las columnas PROVINCIA y MESA_DE_vOTACION
3. Seleccionar todas las columnas excepto DISTRITO.
4. Seleccionar las columnas que comiencen con "VOTOS"
5. Seleccionar las columnas que terminen con "O"
6. Seleccionar las columnas que contengan "_"
7. Seleccionar todas las columnas numéricas
8. Seleccionar las columnas numéricas con algún valor perdido

Función: `pull`

Paquete: `dplyr`

La función `pull` extrae una sola variable de un data frame y la devuelve como vector

¿Cómo se usa? `df |> pull(posición o nombre de la variable a ser seleccionada)`

Ejemplos

1. Seleccionar la variable `DEPARTAMENTO`, como vector.
2. Seleccionar la última variable del data frame, como vector.

En ocasiones, algunos datos pueden estar expresados en una escala que no corresponde a nuestros requerimientos o deseamos realizar una conversión conveniente, por ejemplo: crear una columna de datos en soles a dólares.

Función: `mutate`

Paquete: `dplyr`

¿Cómo se usa?

```
df |> mutate(nueva_variable = regla para crear la nueva variable)
```

```
df |> mutate(variable_existente = regla para modificar la variable)
```

Ejercicios

1. Cree la variable VOTOS_NO_VALIDOS como la suma de votos blancos, nulos e impugnados.
2. Crear la variable UBICACIÓN, concatenando DEPARTAMENTO, PROVINCIA y DISTRITO, dejando un espacio entre valor y valor.
3. Se dice que una mesa es CONCURRIDA si más del 80% de sus electores asiste a votar. Crear la variable CONCURRIDA que tome los valores SI o NO.
4. Se dice que una mesa es CONCURRIDA si más del 80% de sus electores asiste a votar, mientras que es AUSENTE si menos del 20% asiste a votar. Crear la variable GRADO_CONCURRENCIA, que tome los valores AUSENTE, NORMAL y CONCURRIDA
5. Crear dos nuevas columnas, que contengan el porcentaje de votos obtenido por cada candidato
6. Transformar todas las variables que empiezan con VOTOS, reemplazando los NA por 0

Función: `rename`

Paquete: `dplyr`

¿Cómo se usa?

```
df |> rename(nuevo_nombre = posición de la columna)
```

```
df |> rename(nuevo_nombre = antiguo_nombre)
```

Ejercicios

1. Cambiar el nombre de DEPARTAMENTO a DEPTO y PROVINCIA a PROV
2. Cambiar el nombre de todas las variables a minúsculas
3. Cambiar el nombre de DEPARTAMENTO y PROVINCIA a minúsculas
4. Cambiar el nombre de las variables numéricas agregándole NUM delante

Función: `filter`

Paquete: `dplyr`

¿Cómo se usa?

```
df |> filter(reglas de filtro)
```

Operadores lógicos utilizados con `filter`:

- ▶ `<`: menor que
- ▶ `>`: mayor que
- ▶ `==`: igual (un valor)
- ▶ `\texttt{%in%}`: pertenencia a un conjunto
- ▶ `&`: y
- ▶ `|`: o

Ejercicios

1. Filtrar las mesas del departamento de AMAZONAS
2. Filtrar las mesas con más de 250 electores hábiles
3. Filtrar las mesas donde los votos nulos fueron diferentes de cero
4. Filtrar las mesas de Bagua con más de 250 electores hábiles
5. Filtrar las mesas donde P2 obtuvo más votos que P1 y hubo menos de 10 votos nulos
6. Filtrar las mesas donde P1 o P2 obtuvo más de 150 votos
7. Filtrar las mesas de la provincia de LIMA o HUARAL
8. Filtrar las mesas que tienen entre 200 y 300 electores hábiles
9. Filtrar las mesas donde ambas variables de votos blancos o nulos tienen datos
10. Filtrar las mesas donde P1 obtuvo más del 50% de los votos emitidos
11. Filtrar las mesas donde existe una inconsistencia entre los votos y el número de ciudadanos que votaron

Función: arrange

Paquete: dplyr

¿Cómo se usa?

```
df |> arrange(variable) # orden alfabético A Z, o de menor a mayor
```

```
df |> arrange(desc(variable)) # orden alfabético Z A, o de mayor a menor
```

```
df |> arrange(variable numérica) # orden de mayor a menor
```

Ejercicios

1. Ordenar las mesas de menor a mayor cantidad de electores hábiles
2. Ordenar las mesas de mayor a menor cantidad de electores hábiles
3. Ordenar las mesas alfabéticamente según la provincia
4. Ordenar por departamento, provincia y distrito
5. Ordenar por distrito y, dentro de cada distrito, de mayor a menor cantidad de votos de P1

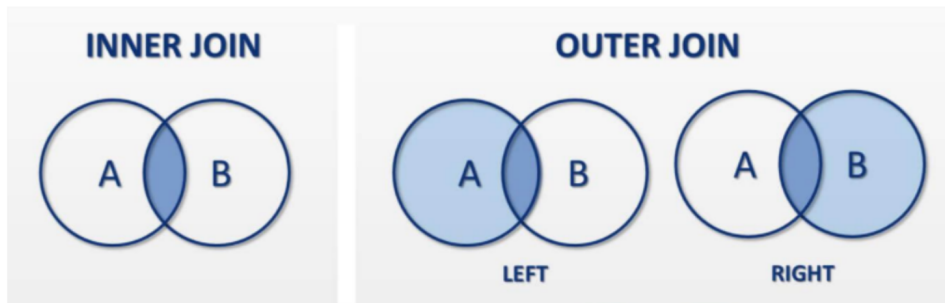
Unión y reorganización de bases de datos

En el análisis de datos es frecuente trabajar con información distribuida en distintas tablas cuya estructura no es adecuada para el análisis. Para resolver estas situaciones se utilizan principalmente dos tipos de operaciones:

- ▶ Integración de datos (o tablas), mediante las funciones `*_join()` de dplyr.
- ▶ Reorganización de datos, mediante las funciones `pivot_longer()` y `pivot_wider()` de tidyr.

Funciones para integración de datos

En ocasiones, nuestros datos no están presentes en un único data frame y necesitamos juntar dos o más de estos.



- ▶ `inner_join(x,y)` retorna las filas de x que tienen valores coincidentes en y, considerando todas las columnas de x e y
- ▶ `right_join(x,y)` retorna las filas de y, además de todas las columnas de x e y. Completa con NA cuando es necesario.
- ▶ `left_join(x,y)` retorna las filas de x, además de todas las columnas de x e y. Completa con NA cuando es necesario.
- ▶ `full_join(x,y)` retorna todas las filas de x e y, hayan o no coincidencias. Completa con NA cuando es necesario.

inner_join(x,y)

x %>% inner_join(y)

x |> inner_join(y)

```
df1 <- data.frame(ID = c(156, 224, 984),  
                  Nombre = c("Ana", "Luis", "María"))  
df2 <- data.frame(ID = c(238, 224, 156),  
                  Ciudad = c("Lima", "Cusco", "Arequipa"))  
df1
```

	ID	Nombre
1	156	Ana
2	224	Luis
3	984	María

df2

	ID	Ciudad
1	238	Lima
2	224	Cusco
3	156	Arequipa

```
df1
```

	ID	Nombre
1	156	Ana
2	224	Luis
3	984	María

```
df2
```

	ID	Ciudad
1	238	Lima
2	224	Cusco
3	156	Arequipa

```
df1 |> inner_join(df2)
```

	ID	Nombre	Ciudad
1	156	Ana	Arequipa
2	224	Luis	Cusco

```
df1
```

	ID	Nombre
1	156	Ana
2	224	Luis
3	984	María

```
df2
```

	ID	Ciudad
1	238	Lima
2	224	Cusco
3	156	Arequipa

```
df1 |> left_join(df2)
```

	ID	Nombre	Ciudad
1	156	Ana	Arequipa
2	224	Luis	Cusco
3	984	María	<NA>

```
df1
```

	ID	Nombre
1	156	Ana
2	224	Luis
3	984	María

```
df2
```

	ID	Ciudad
1	238	Lima
2	224	Cusco
3	156	Arequipa

```
df1 |> right_join(df2)
```

	ID	Nombre	Ciudad
1	156	Ana	Arequipa
2	224	Luis	Cusco
3	238	<NA>	Lima

```
df1
```

	ID	Nombre
1	156	Ana
2	224	Luis
3	984	María

```
df2
```

	ID	Ciudad
1	238	Lima
2	224	Cusco
3	156	Arequipa

```
df1 |> full_join(df2)
```

	ID	Nombre	Ciudad
1	156	Ana	Arequipa
2	224	Luis	Cusco
3	984	María	<NA>
4	238	<NA>	Lima

Funciones para pivoteo de datos

Pivoteo se refiere al proceso de transformar la estructura de un conjunto de datos, reorganizando cómo se distribuyen las filas y columnas.

Hay dos grandes movimientos:

- ▶ `pivot_longer`: Convierte columnas en filas (hace el dataset “más largo”)
- ▶ `pivot_wider`: Convierte filas en columnas (hace el dataset “más ancho”)

Se necesita cargar el paquete `tidyr`. Más información sobre este paquete aquí.

```
library(tidyr)

df <- data.frame(Nombre = c("Ana", "Luis", "María"),
                  Matematica = c(15, 18, 14),
                  Historia = c(17, 16, 19))
```

```
df
```

	Nombre	Matematica	Historia
1	Ana	15	17
2	Luis	18	16
3	María	14	19

```
df_long <- pivot_longer(df,  
                        cols = c(Matematica, Historia),  
                        names_to = "Curso",  
                        values_to = "Nota")  
  
df_long
```

```
# A tibble: 6 x 3
```

	Nombre	Curso	Nota
	<chr>	<chr>	<dbl>
1	Ana	Matematica	15
2	Ana	Historia	17
3	Luis	Matematica	18
4	Luis	Historia	16
5	María	Matematica	14
6	María	Historia	19

```
df_wide <- pivot_wider(df_long,  
                        names_from = Curso,  
                        values_from = Nota)
```

```
df_wide
```

```
# A tibble: 3 x 3
```

	Nombre	Matematica	Historia
	<chr>	<dbl>	<dbl>
1	Ana	15	17
2	Luis	18	16
3	María	14	19

Manejo de fechas y horas con lubridate

El paquete lubridate facilita la creación, transformación, extracción y cálculo de fechas y horas en R.

Las fechas están compuestas por distintos elementos: año, mes, día, hora, minuto, segundo. Además, se pueden escribir de distintas maneras, por ejemplo:

- ▶ 15/07/2026
- ▶ 2026-07-15
- ▶ Julio 15, 2026
- ▶ 15-07-2026 19:00

Funciones para definir fechas: `dmy`, `ymd`, `mdy`.

Funciones para extraer componentes de una fecha: `year`, `month`, `day`, `yday`, `wday`, `week`, `quarter`, `hour`, `minute`, `second`

Ejercicios

1. Convertir 2026-07-15 a formato fecha
2. Convertir 15/Jul/2026 a formato fecha
3. Convertir 07.15.2026 a formato fecha
4. Las funciones `dmy`, `ymd` y `mdy` pueden utilizarse dentro de `mutate`. Por ejemplo, explicar lo que ejecuta este código:

```
datos <- datos |> mutate(Fecha_ingreso = dmy(Fecha_ingreso))
```

5. Almacenar la fecha actual en un objeto de nombre `hoy`.
6. Almacenar "2026-07-16 20:01:19" en formato de fecha y hora.
7. Actualizar el mes de la fecha de la pregunta anterior, de modo que sea diciembre. Almacenar el resultado en `fecha7`.
8. Actualizar la hora de la fecha de la pregunta anterior, de modo que sea 19:18:12. Almacenar el resultado en `fecha8`.

9. Almacenar la fecha y hora actual en un objeto de nombre `ahora`.
10. Añadir 10 días a la fecha actual y almacenar en `fecha10`.
11. Añadir 3 horas a `fecha10` y almacenar en `fecha11`.
12. Añadir un año, 3 meses y 5 horas a `fecha11` y almacenar en `fecha12`.
13. ¿Cuántos días faltan para Navidad?
14. ¿El próximo año será bisiesto?
15. ¿Hoy nos encontramos en un año bisiesto?

Tarea 1